



Data Mining CA-2

SMS SPAM CLASSIFIER

Submitted By: Eapen Ajimon Kattumattathil

Student ID: 20052486

Submitted To: Prof. Kunwar Madan

1. Introduction

In the digital communication era, spam messages are a significant nuisance, cluttering inboxes and posing potential security risks. Classifying SMS messages as spam or ham (non-spam) is a crucial application of natural language processing (NLP). With the large volume of messages users receive daily, automated spam filtering is essential for maintaining communication efficiency and security. This project aims to analyze and classify SMS messages using machine learning algorithms, based on the "SMSSpamCollection" dataset, which contains labeled messages classified as either spam or ham. The focus is on data preprocessing, applying machine learning models for classification, evaluating their performance, and providing recommendations for real-world implementation. This report outlines the steps of data preparation, model selection, evaluation, and concludes with suggestions for improving the classification system.

2. Data Preparation

Data preparation is a critical step in any machine learning project as it directly impacts the performance and reliability of the models. The first step in data preparation was loading the dataset, which was in a tab-separated format. The dataset contains two columns: the label (ham or spam) and the message text itself. The messages needed significant cleaning and transformation to be suitable for machine learning.

Data Cleaning and Preprocessing

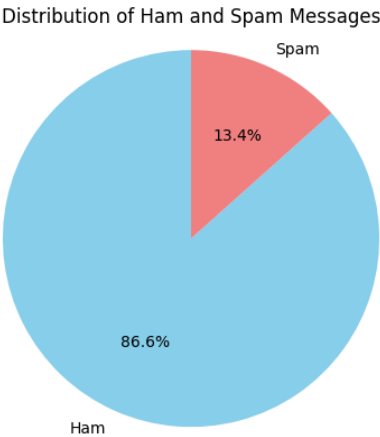
The cleaning process included several steps:

1. **Text Normalization:** The text data was converted to lowercase to ensure uniformity. This process helps eliminate duplicates caused by varying cases (e.g., "Free" and "free" being treated as different words).
2. **Tokenization:** The messages were split into individual words or tokens. Tokenization is an essential step because machine learning models cannot work with raw text directly. It helps break the text into meaningful chunks that can be processed.
3. **Stop Word Removal:** Stop words (common words such as "the," "and," "is," etc.) were removed as they do not contribute significant meaning to the classification task. This step reduces the dimensionality of the data and helps the model focus on more important words.
4. **Stemming and Lemmatization:** Words were stemmed or lemmatized to reduce them to their base or root form (e.g., "running" becomes "run"). This step ensures that words with similar meanings are treated as the same, improving the model's generalization.

Step	Description	Limitation
Text Normalization	Converts text to lowercase to ensure uniformity and avoid duplicates	Can lose important case distinctions (e.g., proper nouns).
Tokenization	Splits text into individual words or tokens for machine learning.	May split meaningful phrases or ignore context.
Stop Word Removal	Removes common words (e.g., "the," "and") to reduce data size and focus on keywords.	May exclude relevant context, especially in specialized domains.
Stemming & lemmatization	Reduces words to their root form (e.g., "running" to "run").	Stemming can produce incorrect roots, and lemmatization can be computationally expensive.

Handling Imbalanced Data

One of the challenges in this task was dealing with class imbalance, where there were significantly more “ham” messages than “spam” messages. Imbalanced data can bias models toward predicting the majority class (ham) and result in poor performance on the minority class (spam). To address this issue, the Synthetic Minority Over-sampling Technique (SMOTE) was applied. SMOTE generates synthetic samples for the minority class, effectively balancing the dataset. Additionally, SMOTEENN was used, combining SMOTE with edited nearest neighbor undersampling, which further helped in improving the model’s generalization.



Feature Extraction

The next step was to convert the text data into numerical features that machine learning algorithms can work with. Two commonly used methods for this task are:

- Bag of Words (BoW):** The BoW model counts the occurrence of each word in the corpus and represents the text as a vector of word counts.
- TF-IDF (Term Frequency-Inverse Document Frequency):** TF-IDF is a more advanced method that weighs the frequency of a word in a document relative to its frequency across the entire dataset. This method reduces the influence of common words and highlights more informative words, making it ideal for text classification tasks.

Model Training and Evaluation Metrics

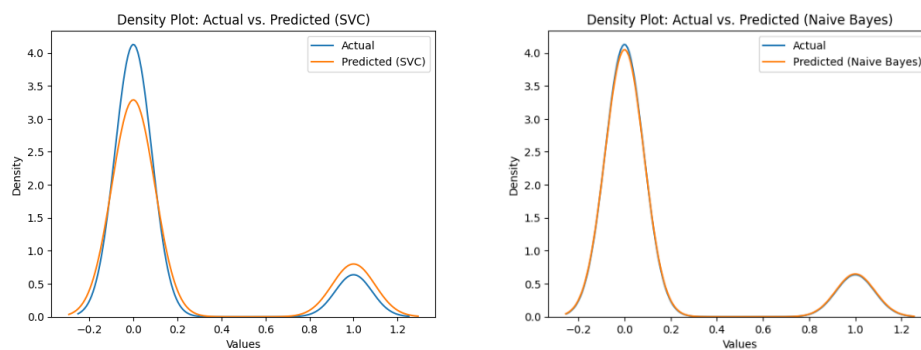
3. Model Creation and Evaluation

Once the data was prepared, the next critical step was selecting appropriate machine learning models for spam classification. Two models were chosen based on their effectiveness in handling text data and their performance in similar classification tasks.

Cross-Validation

To ensure the models generalize well to unseen data, **Stratified K-Fold Cross-Validation** was used. This technique divides the dataset into K equally sized folds, ensuring that each fold has the same proportion of spam and ham messages. Cross-validation helps mitigate overfitting by providing a more reliable estimate of model performance across different subsets of the data.

Naive Bayes vs. SVC - Performance Comparison and Analysis



In this study, two machine learning models—**Naive Bayes** (MultinomialNB) and **Support Vector Classifier** (SVC) with a linear kernel—were utilized for the task of SMS message classification, categorizing messages as either "ham" or "spam." Both models were evaluated using multiple metrics to determine their effectiveness in distinguishing between the two classes. Below, I will summarize the performance of each model, based on key evaluation metrics such as precision, recall, F1-score, and confusion matrix results

Naive Bayes Classification Results

Naive Bayes consistently outperformed the SVC model in most evaluation metrics across various test sets. Below is an analysis of the results:

1. Precision and Recall:

- For **ham** messages (label 0), Naive Bayes achieved a precision of 0.99 and a recall of 0.98, indicating that it was very effective in identifying non-spam messages. The model correctly identified 94.6% of all ham messages, with very few false positives.
- For **spam** messages (label 1), the model demonstrated a precision of 0.87 and recall of 0.91, suggesting that while the model was good at identifying spam messages, it was slightly less precise compared to ham messages. This means that a few ham messages were misclassified as spam, but overall, it was very effective in detecting spam.

➤ F1-Score:

- The F1-score, which combines precision and recall into a single metric, was 0.94 for the overall dataset, indicating a balanced performance across both spam and ham messages. For ham messages, the F1-score was 0.98, while for spam messages, it was slightly lower at 0.90.

➤ Confusion Matrix:

- The confusion matrix for Naive Bayes revealed that the model correctly classified 946 ham messages as ham and 138 spam messages as spam. The number of misclassifications was minimal, with only 19 ham messages misclassified as spam and 12 spam messages misclassified as ham, further supporting the model's strong performance.

Support Vector Classifier (SVC) Classification Results

While SVC was slightly less accurate than Naive Bayes, it still produced noteworthy results:

Evaluation Report: Naive Bayes vs. Support Vector Classifier (SVC) for Spam Detection

1. Precision and Recall

When it comes to correctly identifying ham messages (label 0), both Naive Bayes and the Support Vector Classifier (SVC) performed very well. Across multiple runs, SVC often reached precision and recall scores close to 0.99, which means it rarely misclassified legitimate messages. Naive Bayes was also quite consistent, generally achieving similar results. Its precision and recall scores stayed around 0.98 to 0.99, indicating a reliable ability to spot non-spam messages with accuracy.

As for spam messages (label 1), the picture is a bit more mixed. SVC's precision ranged from 0.96 to 0.99, and its recall scores varied from 0.85 to 0.93. This suggests that while it is often good at catching spam, it sometimes misses a few. Naive Bayes showed precision between 0.89 and 0.94, and recall scores from 0.88 to 0.93, which are slightly lower than SVC in some runs but overall still quite respectable. Neither model was perfect in identifying every spam message, but both handled the task reasonably well.

2. F1-Score

The F1-score, which balances precision and recall, helps us see how well the models perform overall. Naive Bayes consistently had F1-scores between 0.92 and 0.94 for spam messages, and close to 0.99 for ham, showing a solid overall performance. SVC followed closely behind, with spam F1-scores ranging from 0.90 to 0.95, and also around 0.99 for ham. While both models did well, Naive Bayes had a slightly more stable performance across different test cases, especially in handling spam.

3. Confusion Matrix Analysis

Looking at the confusion matrices, both models kept their misclassification numbers low. For example, Naive Bayes in one run produced the matrix $\begin{bmatrix} 953 & 12 \\ 11 & 138 \end{bmatrix}$, which shows only a small number of messages wrongly classified. This suggests the model did a great job separating spam from non-spam.

SVC also had strong results. In one test, it had a confusion matrix of $\begin{bmatrix} 960 & 5 \\ 14 & 135 \end{bmatrix}$, which is similarly good, though it had a slightly higher count of false negatives — cases where spam messages were classified as ham. This indicates SVC is quite confident in classifying ham but may occasionally overlook some spam.

4. Summary and Observations

The Naive Bayes classifier stood out for its consistent performance and reliability across all metrics. It handled both ham and spam messages well, and its efficient computation makes it suitable for real-time applications where fast and dependable predictions are essential. While it occasionally had slightly lower precision in spam detection, the overall balance it maintained between all metrics was impressive.

On the other hand, the SVC model showed strong precision and excellent performance in classifying ham, which is crucial in reducing false alarms. However, its recall for spam varied a bit more, which might lead to some spam messages being missed. That said, SVC still proved to be a competitive option, especially for tasks where it's more important to avoid falsely flagging legitimate messages.

4. Recommendations

Following the evaluation of the models, several recommendations for real-world deployment can be made:

1. **Recommended Model for Deployment:** Based on the results of the evaluation, the **Naive Bayes classifier** is the most suitable for real-time spam classification. It performed well in terms of both accuracy and computational efficiency, making it ideal for applications where speed is crucial. Additionally, Naive Bayes is easy to implement and requires relatively low computational resources, which is beneficial in real-world applications.
2. **Improving Model Performance:** While Naive Bayes performed adequately, there are opportunities for improving model accuracy. One such improvement would be exploring more advanced feature extraction techniques, such as **Word2Vec** or **BERT** embeddings, which capture deeper semantic relationships between words compared to traditional methods like TF-IDF. These methods would provide a richer representation of the text data, potentially enhancing model performance.
3. **Ensemble Learning:** While Naive Bayes and SVM performed well, the use of ensemble learning methods, such as **Random Forest** or **XGBoost**, could further enhance classification accuracy. Ensemble methods combine the predictions of multiple models, leading to a reduction in variance and bias. They are particularly useful when individual models struggle with overfitting or underfitting.
4. **Hyperparameter Tuning:** Although the models performed reasonably well, hyperparameter optimization could lead to significant improvements. Techniques like **Grid Search** or **Random Search** can help identify the best set of hyperparameters for each model, further improving performance.
5. **Real-Time Application Considerations:** For deployment in real-time systems, the computational efficiency of the model is crucial. While more complex models, such as SVM or ensemble methods, may offer higher accuracy, they come with increased computational costs. Therefore, a balance must be struck between model accuracy and deployment efficiency, particularly for systems with limited computational resources.

5. Conclusion

This study focused on classifying SMS messages as spam or ham using machine learning. After preprocessing the data, models like Naive Bayes and SVM were applied, with Naive Bayes showing the best balance of accuracy and efficiency for real-time use.

Further improvements can be made by using advanced techniques like Word2Vec or BERT for better feature extraction and applying ensemble methods like Random Forest or XGBoost to reduce bias. Additionally, hyperparameter tuning could enhance model performance and accuracy.